

Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики и
радиоэлектроники»

Факультет [УТОЧНИТЬ]
Кафедра [УТОЧНИТЬ]

ОТЧЕТ

по практическому занятию №1
«*Middleware* и шаблонизаторы»
по дисциплине «ИТиВП» [УТОЧНИТЬ полное название]

Выполнил: студент гр. [УТОЧНИТЬ]
[УТОЧНИТЬ ФИО]
Проверил: [УТОЧНИТЬ ФИО
преподавателя]

Минск 2026

1 Цель работы

Целью работы является освоение концепции *middleware* в *Express.js*, разработка собственных *middleware* для логирования, имитации авторизации и обработки ошибок, а также изучение шаблонизатора *EJS* для генерации динамических *HTML*-страниц на стороне сервера. Тематика проекта соответствует курсовой работе: платформа для создания визуальных *DSL*-диаграмм (*VisualDSL*), данные которой хранятся в массиве в памяти.

2 Задание и порядок выполнения

В рамках практического занятия необходимо решить пять задач, каждая из которых получила отражение в отдельном разделе отчета и в коде проекта:

- создать *Express*-приложение с поддержкой *EJS* и настроить *view engine*
- реализовать *middleware* логирования метода, *URL* и времени запроса с сохранением записей в журнал, который выводится в виде таблицы
- реализовать *middleware* имитации авторизации по параметру *?auth=1*
- создать динамические страницы: главную, детальную и форму добавления
- добавить обработку ошибок 404 и 500 с отдельными страницами

Данные хранятся в массиве в оперативной памяти, база данных не используется. Код размещен в ветке *pz21* репозитория проекта.

3 Ход работы

3.1 Настройка проекта и *EJS*

Проект построен на основе серверной части из предыдущей лабораторной работы: точка входа *server.js* запускает приложение, а само приложение собирается в *app.js*. Такое разделение позволяет подключать приложение в автоматических тестах без запуска сетевого порта. Зависимость *ejs* устанавливается командой *npm install ej*s, а в *app.js* задаются параметры *view engine* и *views* (листинг 1). Статические файлы подключаются до маршрутов, чтобы запросы к стилям не проходили через остальную цепочку *middleware*.

Листинг 1 – Настройка *Express* и порядок подключения *middleware* (*app.js*)

```
const path = require('path');
const express = require('express');
const logger = require('./middlewares/logger');
const { attachUser } = require('./middlewares/auth');
const { notFoundHandler, serverErrorHandler } =
  require('./middlewares/errorHandlers');
const pageRoutes = require('./routes/pageRoutes');
const diagramRoutes = require('./routes/diagramRoutes');
```

```

const app = express();

// Шаблонизатор EJS.
app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));

// Helper для шаблонов: форматирование даты.
app.locals.formatDate = (iso) => new Date(iso).toLocaleString('ru-RU');

// Статика подключается до маршрутов, чтобы файлы отдавались без
лишних проверок.
app.use('/static', express.static(path.join(__dirname, 'public')));

app.use(logger);

app.use(express.urlencoded({ extended: true }));
app.use(express.json());
app.use(attachUser);

app.use('/', pageRoutes);
app.use('/api/diagrams', diagramRoutes);

// Обработчики ошибок идут последними.
app.use(notFoundHandler);
app.use(serverErrorHandler);

module.exports = app;

```

3.2 Шаблоны EJS

Общий каркас страницы вынесен в частичные шаблоны *partials/header.ejs* и *partials/footer.ejs*, которые подключаются конструкцией *include* в каждой странице. Благодаря этому разметка `<html>` и шапка сайта не повторяются. Созданы страницы *index.ejs* (список), *item.ejs* (детали), *add.ejs* (форма), *login.ejs*, *404.ejs* и *500.ejs*. Шаблон главной страницы приведен в листинге 2: цикл *forEach* выводит массив диаграмм в таблицу, а вывод через `<%= %>` экранирует HTML.

Листинг 2 – Шаблон главной страницы (*views/index.ejs*)

```

<%- include('partials/header') %>
<h1>Список диаграмм</h1>
<p><a class="btn" href="/add<%= authQuery %>">Добавить
диаграмму</a></p>

<% if (items.length === 0) { %>
  <p class="muted">Диаграмм пока нет.</p>
<% } else { %>
  <table>
    <thead>

```

```

<tr><th>ID</th><th>Название</th><th>Тип</th><th>Создана</th></tr>
</thead>
<tbody>
  <% items.forEach(function (item) { %>
    <tr>
      <td><%= item.id %></td>
      <td><a href="/item/<%= item.id %><%= authQuery
%>"><%= item.title %></a></td>
      <td><span class="badge"><%= item.type
%></span></td>
      <td><%= formatDate(item.createdAt) %></td>
    </tr>
  <% }); %>
</tbody>
</table>
<% } %>
<%- include('partials/footer') %>

```

Листинг 3 – Частичный шаблон шапки (views/partials/header.ejs)

```

<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title><%= title %> | VisualDSL</title>
  <link rel="stylesheet" href="/static/css/style.css">
</head>
<body>
<header class="site-header">
  <a class="brand" href="/<%= authQuery %>">VisualDSL</a>
  <nav>
    <a href="/<%= authQuery %>">Диаграммы</a>
    <a href="/add<%= authQuery %>">Добавить</a>
    <a href="/logs<%= authQuery %>">Журнал</a>
  </nav>
  <span class="user">Пользователь: <strong><%= user.name %></strong>
    <% if (!user.isAuthenticated) { %><a href="/login">Войти</a><% } %>
  </span>
</header>
<main class="container">

```

3.3 Middleware

Логирующее *middleware* (листинг 4) подписывается на событие *finish* ответа, выводит в консоль метод, *URL*, код состояния и время обработки в миллисекундах, а также сохраняет запись в журнал *logStore* (листинг 5). Запись содержит время, тип события (успешный запрос, перенаправление, ошибка клиента или сервера), метод, *URL*, статус, длительность, имя пользователя и *IP*-адрес; журнал ограничен 500 записями и выводится таблицей на странице */logs*. *Middleware* авторизации (листинг 6) состоит из

двух функций: *attachUser* определяет пользователя по параметру *?auth=1* и передает его в шаблоны через *res.locals*, а *requireAuth* перенаправляет гостя на */login*. Обе стратегии, допустимые заданием, реализованы одновременно: гость получает *req.user* со значением «Гость», а защищенные маршруты требуют авторизацию.

Листинг 4 – *Middleware* логирования (*middlewares/logger.js*)

```
// Логирование: метод, URL, статус и время обработки (мс) после
отправки ответа.
// Запись попадает в журнал (таблица на странице /logs) и в консоль
(кроме тестов).
const logStore = require('../models/logStore');

module.exports = (req, res, next) => {
  const start = process.hrtime.bigint();

  res.on('finish', () => {
    const ms = Number(process.hrtime.bigint() - start) / 1e6;
    const entry = logStore.add({
      method: req.method,
      url: req.originalUrl,
      status: res.statusCode,
      durationMs: ms,
      user: req.user ? req.user.name : 'Гость',
      ip: req.ip || '-'
    });

    if (process.env.NODE_ENV !== 'test') {
      console.log(
        `[${entry.time}] ${entry.method} ${entry.url} ->
        ${entry.status} (${entry.durationMs.toFixed(2)} мс)`
      );
    }
  });

  next();
};
```

Листинг 5 – Хранилище журнала запросов (*models/logStore.js*)

```
// Журнал запросов в памяти: записи отображаются на странице /logs в
виде таблицы.
const MAX_ENTRIES = 500;

let entries = [];
let nextId = 1;

// Тип события определяется по коду ответа.
const eventByStatus = (status) => {
  if (status >= 500) return 'Ошибка сервера';
  if (status >= 400) return 'Ошибка клиента';
  if (status >= 300) return 'Перенаправление';
  return 'Успешный запрос';
};
```

```

};

const add = ({ method, url, status, durationMs, user, ip }) => {
  const entry = {
    id: nextId++,
    time: new Date().toISOString(),
    event: eventByStatus(status),
    method,
    url,
    status,
    durationMs: Number(durationMs.toFixed(2)),
    user,
    ip
  };
  entries.push(entry);
  if (entries.length > MAX_ENTRIES) entries.shift();
  return entry;
};

// Новые записи первыми; необязательные фильтры по методу и классу
события.
const getAll = ({ method, event } = {}) =>
  entries
    .filter((e) => (!method || e.method === method) && (!event ||
e.event === event))
    .reverse();

const clear = () => {
  entries = [];
  nextId = 1;
};

module.exports = { MAX_ENTRIES, eventByStatus, add, getAll, clear };

```

Листинг 6 – *Middleware* имитации авторизации (*middlewares/auth.js*)

```

// Имитация авторизации по query-параметру ?auth=1 (без сессий и
паролей).
const GUEST = { name: 'Гость', isAuth: false };
const ADMIN = { name: 'Администратор', isAuth: true };

// Определяет пользователя и передает его в шаблоны (res.locals).
const attachUser = (req, res, next) => {
  req.user = req.query.auth === '1' ? ADMIN : GUEST;
  res.locals.user = req.user;
  // Суффикс для ссылок, чтобы "авторизация" сохранялась при переходах
по сайту.
  res.locals.authQuery = req.user.isAuth ? '?auth=1' : '';
  next();
};

// Защита маршрута: гость перенаправляется на /login.
const requireAuth = (req, res, next) => {

```

```

    if (req.user && req.user.isAuthenticated) return next();
    res.redirect('/login');
  };

```

```

module.exports = { attachUser, requireAuth };

```

3.4 Маршруты и рендеринг

Маршруты страниц описаны в *routes/pageRoutes.js* (листинг 7), а логика обработчиков находится в *controllers/pageController.js*. Список и детальная страница доступны всем, маршруты */add* защищены *middleware requireAuth*. Данные берутся из общего хранилища *models/diagramStore.js*, которое используется и страницами, и *REST API*, поэтому добавленная через форму диаграмма сразу видна в *API*. Форма проверяет поля на стороне сервера и при ошибке повторно показывает введенные значения с кодом 400.

Листинг 7 – Маршруты страниц (*routes/pageRoutes.js*)

```

const express = require('express');
const { requireAuth } = require('../middlewares/auth');
const pages = require('../controllers/pageController');

const router = express.Router();

router.get('/', pages.index);
router.get('/item/:id', pages.item);
router.get('/login', pages.login);
router.get('/logs', pages.logs);
// Middleware requireAuth выполняется только для этих маршрутов.
router.get('/add', requireAuth, pages.addForm);
router.post('/add', requireAuth, pages.addSubmit);

if (process.env.NODE_ENV !== 'production') {
  router.get('/debug/error', pages.crash);
}

module.exports = router;

```

3.5 Обработка ошибок

Обработчики ошибок (листинг 8) подключаются последними. Функция *notFoundHandler* отображает страницу *404.ejs*, а *serverErrorHandler* имеет четыре параметра, что делает ее обработчиком ошибок *Express*: она пишет стек в консоль и отображает *500.ejs*. Для запросов к */api/* обе функции возвращают *JSON*, поэтому клиенты *API* не получают *HTML*. Для демонстрации страницы 500 добавлен маршрут */debug/error*, отключаемый при *NODE_ENV=production*.

Листинг 8 – Обработчики ошибок (*middlewares/errorHandlers.js*)

```

// Обработчики ошибок: 404 (маршрут не найден) и 500 (исключение в
приложении).
const wantsJson = (req) => req.originalUrl.startsWith('/api/');

```

```
const notFoundHandler = (req, res) => {
  if (wantsJson(req)) return res.status(404).json({ error: 'Маршрут не найден' });
  res.status(404).render('404', { title: 'Ошибка 404' });
};

// Четыре параметра - признак error-handling middleware в Express.
const serverErrorHandler = (err, req, res, next) => {
  console.error(err.stack || err);
  if (res.headersSent) return next(err);
  if (wantsJson(req)) {
    return res.status(500).json({ error: 'Внутренняя ошибка сервера',
      message: err.message });
  }
  res.status(500).render('500', { title: 'Ошибка 500' });
};

module.exports = { notFoundHandler, serverErrorHandler };
```

3.6 Результаты работы приложения

Для проверки приложение запущено командой `npm start`, страницы открыты в браузере. Главная страница с таблицей диаграмм из массива показана на рисунке 1.

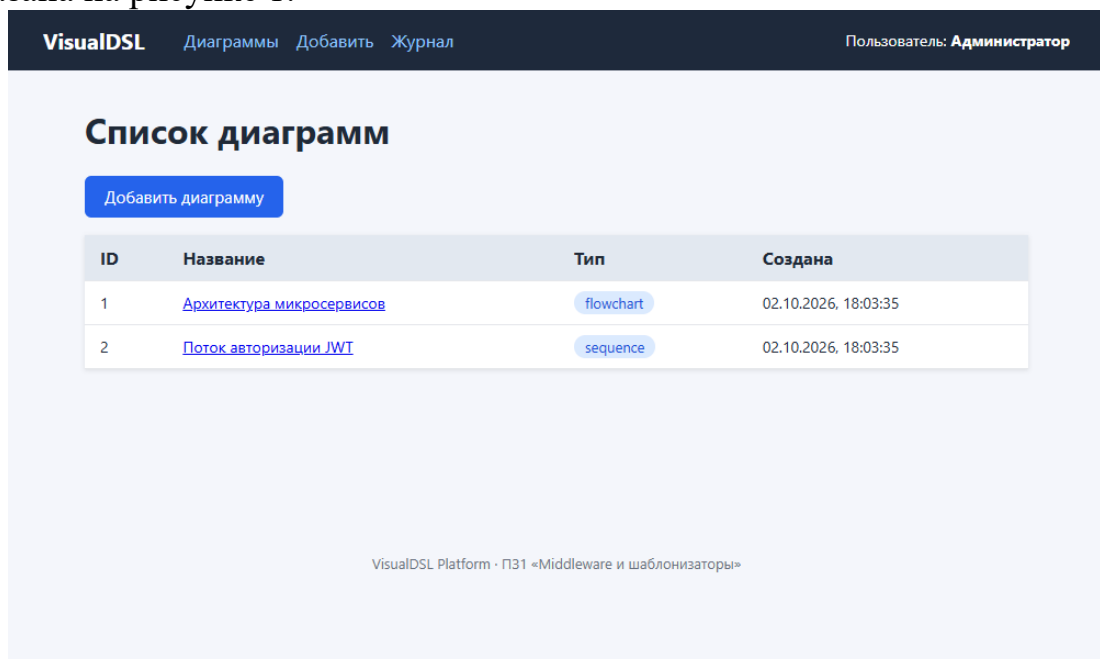


Рисунок 1 – Главная страница со списком диаграмм

На рисунке 1 видны два начальных элемента массива, а имя пользователя в шапке и параметр `?auth=1` в ссылках подтверждают работу *middleware* авторизации. Цикл `forEach` в шаблоне `index.ejs` вывел каждый элемент отдельной строкой таблицы.

Страница на рисунке 2 открывается по адресу `/item/1` и выводит поля выбранного элемента.

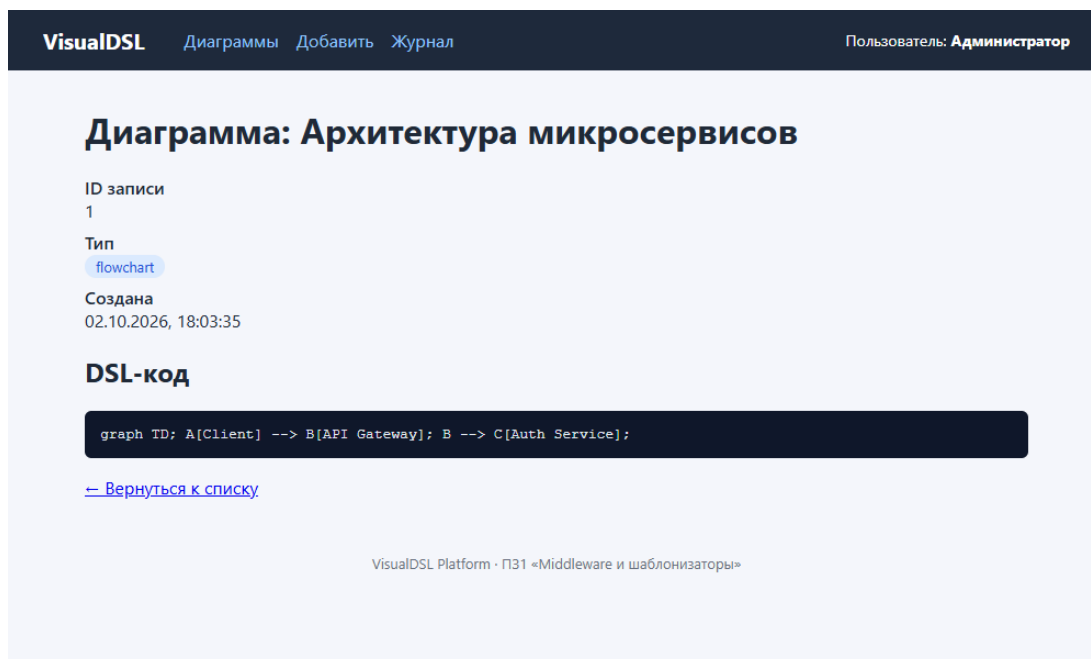


Рисунок 2 – Детальная страница диаграммы

Маршрут `/item/:id` нашел элемент по идентификатору и передал его в шаблон `item.ejs`. DSL-код отображается в экранированном виде, поэтому разметка внутри кода не может выполняться в браузере.

Форма добавления доступна только после авторизации и показана на рисунке 3.

Рисунок 3 – Форма добавления диаграммы

Форма содержит поля названия, типа и DSL-кода. Адрес отправки `/add?auth=1` сохраняет признак авторизации, поэтому после отправки пользователь остается в системе.

Заполненная форма с данными новой диаграммы приведена на рисунке 4.

VisualDSL

ДиаграммыДобавитьЖурнал

Пользователь: Администратор

Новая диаграмма

Название

Диаграмма состояний заказа

Тип диаграммы

state

DSL-код

stateDiagram-v2; [*] --> New; New --> Paid; Paid --> [*];

Добавить

Отмена

Рисунок 4 – Заполненная форма

Введены название «Диаграмма состояний заказа», тип *state* и *DSL*-код. После нажатия кнопки данные уходят методом *POST* на сервер, где проходят проверку.

Результат отправки показан на рисунке 5: сервер выполнил перенаправление на главную страницу.

VisualDSL

ДиаграммыДобавитьЖурнал

Пользователь: Администратор

Список диаграмм

Добавить диаграмму

ID	Название	Тип	Создана
1	Архитектура микросервисов	flowchart	02.10.2026, 18:03:35
2	Поток авторизации JWT	sequence	02.10.2026, 18:03:35
3	Диаграмма состояний заказа	state	02.10.2026, 18:03:38

VisualDSL Platform · ПЗ1 «Middleware и шаблонизаторы»

Рисунок 5 – Список после добавления элемента

Новая диаграмма появилась в списке третьей строкой. Это подтверждает, что *POST /add* добавил элемент в массив, а перенаправление исключает повторную отправку формы при обновлении страницы.

При отправке пустой формы сервер отвечает кодом 400 и показывает список ошибок (рисунок 6).

Рисунок 6 – Сообщения валидации формы

Проверка выполняется на стороне сервера: пустые поля названия и DSL-кода отклонены, а введенные значения сохранены в форме. Массив данных при этом не изменился.

Гость, открывший `/add` без параметра `auth`, попадает на страницу `/login` (рисунок 7).

Рисунок 7 – Перенаправление гостя на страницу авторизации

`Middleware requireAuth` остановил запрос и выполнил перенаправление. Страница предлагает войти как администратор через `?auth=1` или продолжить как гость.

Несуществующий адрес приводит к странице 404 (рисунок 8).

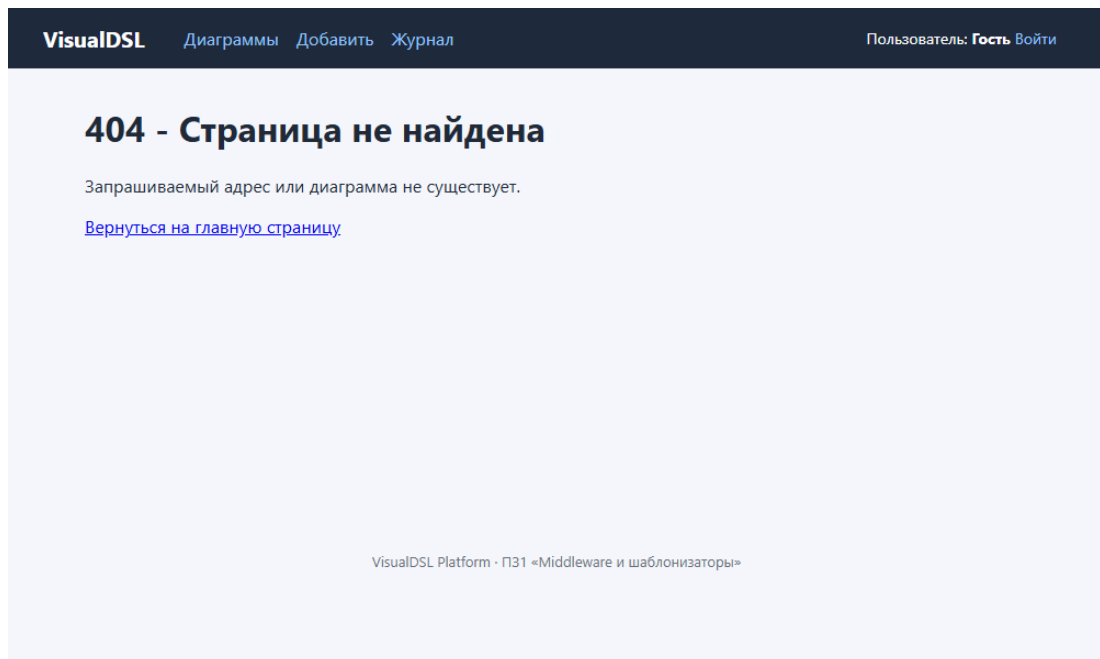


Рисунок 8 – Страница ошибки 404

Обработчик *notFoundHandler* сработал, потому что ни один маршрут не ответил на запрос, и отобразил отдельный шаблон *404.ejs* с ссылкой на главную страницу.

Намеренно вызванное исключение на маршруте */debug/error* приводит к странице 500 (рисунок 9).

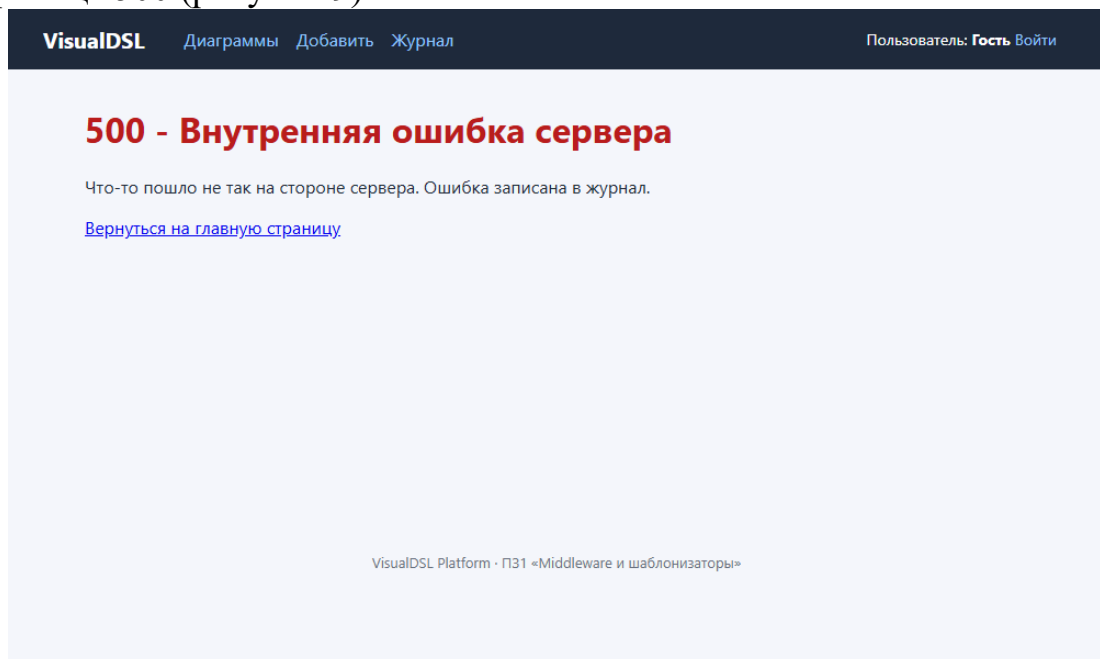


Рисунок 9 – Страница ошибки 500

Обработчик ошибок *serverErrorHandler* записал стек в консоль и показал пользователю безопасную страницу *500.ejs* без технических подробностей.

Журнал запросов, сохраненный логирующим *middleware*, показан таблицей на странице */logs* (рисунок 10).

VisualDSL

Диаграммы

Добавить

Журнал

Пользователь: Администратор

Журнал запросов

Всего записей: 13. Новые записи сверху.

Все методы

Все события

Применить

№	Время	Событие	Метод	URL	Статус	Время, мс	Пользователь	IP
13	02.10.2026, 18:03:38	Ошибка сервера	GET	/debug/error	500	2.05	Гость	::1
12	02.10.2026, 18:03:38	Ошибка клиента	GET	/no-such-page	404	1.87	Гость	::1
11	02.10.2026, 18:03:38	Успешный запрос	GET	/login	200	2.04	Гость	::1
10	02.10.2026, 18:03:38	Перенаправление	GET	/add	302	0.63	Гость	::1
9	02.10.2026, 18:03:38	Ошибка клиента	POST	/add?auth=1	400	1.76	Администратор	::1
8	02.10.2026, 18:03:38	Перенаправление	POST	/add?auth=1	302	11.46	Администратор	::1

Рисунок 10 – Журнал запросов в виде таблицы (страница /logs)

В таблице видны время, событие, метод, *URL*, статус, длительность, пользователь и *IP*-адрес. Запросы разных видов отличаются типом события: успешный запрос, перенаправление, ошибка клиента и ошибка сервера. Новые записи расположены сверху.

Таблицу можно отфильтровать по методу и типу события; на рисунке 11 оставлены только запросы *POST*.

VisualDSL

Диаграммы

Добавить

Журнал

Пользователь: Администратор

Журнал запросов

Всего записей: 2. Новые записи сверху.

POST

Все события

Применить

№	Время	Событие	Метод	URL	Статус	Время, мс	Пользователь	IP
9	02.10.2026, 18:03:38	Ошибка клиента	POST	/add?auth=1	400	1.76	Администратор	::1
6	02.10.2026, 18:03:38	Перенаправление	POST	/add?auth=1	302	11.46	Администратор	::1

VisualDSL Platform · ПЗ1 «Middleware и шаблонизаторы»

Рисунок 11 – Журнал с фильтром по методу *POST*

Фильтр передается параметрами строки запроса, а сервер проверяет их по списку допустимых значений, поэтому произвольный ввод игнорируется.

Те же записи выводятся в консоль сервера (рисунок 12), что удобно при отладке.

```

Сервер запущен на http://localhost:5055
[2026-10-02T15:03:36.710Z] GET /?auth=1 -> 200 (33.92 мс)
[2026-10-02T15:03:36.765Z] GET /favicon.ico -> 404 (3.02 мс)
[2026-10-02T15:03:37.943Z] GET /item/1?auth=1 -> 200 (2.56 мс)
[2026-10-02T15:03:38.028Z] GET /add?auth=1 -> 200 (2.49 мс)
[2026-10-02T15:03:38.177Z] GET /add?auth=1 -> 304 (2.08 мс)
[2026-10-02T15:03:38.313Z] POST /add?auth=1 -> 302 (11.46 мс)
[2026-10-02T15:03:38.317Z] GET /?auth=1 -> 200 (1.83 мс)
[2026-10-02T15:03:38.390Z] GET /add?auth=1 -> 200 (1.94 мс)
[2026-10-02T15:03:38.461Z] POST /add?auth=1 -> 400 (1.76 мс)
[2026-10-02T15:03:38.519Z] GET /add -> 302 (0.63 мс)
[2026-10-02T15:03:38.523Z] GET /login -> 200 (2.04 мс)
[2026-10-02T15:03:38.586Z] GET /no-such-page -> 404 (1.87 мс)
Error: Тестовая ошибка для демонстрации страницы 500
[2026-10-02T15:03:38.643Z] GET /debug/error -> 500 (2.05 мс)
[2026-10-02T15:03:38.710Z] GET /logs?auth=1 -> 200 (2.51 мс)
[2026-10-02T15:03:38.792Z] GET /logs?auth=1&method=POST -> 200 (2.14 мс)

```

Рисунок 12 – Журнал запросов в консоли сервера

Каждая строка содержит время, метод, *URL*, код ответа и длительность обработки в миллисекундах. Сообщение «*Error*» перед последней строкой появилось при обращении к */debug/error* и подтверждает, что ошибка сервера записывается в журнал.

3.7 Автоматическое тестирование

Для проверки приложения написаны автоматические тесты на *Jest* и *Supertest*. Они вызывают приложение без запуска сервера и проверяют коды ответов, содержимое *HTML*, редиректы, экранирование и работу каждого *middleware* по отдельности. Результаты приведены в таблице 1; команда запуска: *npm test*.

Таблица 1 – Результаты автоматического тестирования

Набор тестов	Что проверяется	Тестов
<i>api.test.js</i>	<i>REST API /api/diagrams: CRUD</i> , поиск, валидация, <i>JSON</i> -ошибки	25
<i>pages.test.js</i>	<i>EJS</i> -страницы, авторизация, форма, журнал <i>/logs</i> , 404 и 500, статика, <i>XSS</i>	30
<i>middleware.test.js</i>	<i>logger</i> , <i>logStore</i> , <i>attachUser</i> , <i>requireAuth</i> , обработчики ошибок, хранилище	21
<i>seeds.test.js</i>	Начальные данные, фильтры журнала, <i>production</i> , запуск <i>server.js</i> , сквозной сценарий	14
<i>branches.test.js</i>	Редкие ветви: пустое тело запроса, ошибка без стека	5
<i>extras/__tests__</i>	Заготовки для защиты: блокировка <i>IP</i> , <i>CSRF</i> , поиск, редактирование и другие	14
<i>src/__tests__</i>	Тесты <i>React</i> -компонентов предыдущих работ	8
Итого	Все наборы пройдены успешно	117

Покрытие серверного кода, измеренное командой *npm run test:coverage*, составило 100 % по строкам, функциям и операторам и 96,5 % по ветвям. Непокрытыми остаются только запасные выражения *req.body || {}*, которые

недостижимы, так как парсеры *Express* всегда создают объект *req.body*. Начальные данные для тестов и демонстрации вынесены в папку *seeds/*: массив диаграмм и набор записей журнала, содержащий все типы событий и методы. Для показа на защите добавлена команда *npm run demo*, которая отправляет на запущенный сервер запросы всех видов и наполняет таблицу журнала.

4 Ответы на контрольные вопросы

1. Что такое *middleware* в *Express.js* и какова его сигнатура?

Middleware – это функция, которая получает объекты запроса и ответа и управляет дальнейшей обработкой. Сигнатура: (*req*, *res*, *next*). Функция может изменить *req* и *res*, завершить ответ или передать управление вызовом *next()*.

2. Какой порядок выполнения *middleware* в *Express*? Что влияет на порядок?

Middleware выполняются строго в порядке регистрации через *app.use()* и *app.METHOD()*. На порядок влияет место вызова в коде, а также совпадение пути и метода: функция выполняется, только если запрос подходит под ее путь.

3. Как передать данные из маршрута в шаблон *EJS*?

Данные передаются вторым аргументом *res.render()*: *res.render('index', { items: diagrams })*. В шаблоне они доступны как переменные, например `<%= items.length %>`. Общие данные можно положить в *res.locals* или *app.locals*.

4. Что такое *next()* и что произойдет, если его не вызвать?

Функция *next()* передает управление следующему *middleware*. Если ее не вызвать и не отправить ответ, запрос зависнет, и клиент будет ждать ответа до таймаута. Вызов *next(err)* передает управление обработчику ошибок.

5. Как создать *middleware* для обработки ошибок?

Нужно объявить функцию с четырьмя параметрами (*err*, *req*, *res*, *next*) и подключить ее после всех маршрутов. *Express* определяет обработчик ошибок именно по числу параметров. В проекте это *serverErrorHandler*.

6. В чем разница между *app.use()* и *app.METHOD()*?

Метод *app.use()* подключает *middleware* для всех *HTTP*-методов и для любого пути, начинающегося с указанного префикса. Метод *app.get()* и аналоги срабатывают только для своего *HTTP*-метода и точного совпадения пути.

7. Как в *EJS* использовать циклы и условные операторы?

Управляющий код записывается в тегах `<% %>`. Пример: `<% if (items.length) { %> <ul> <% items.forEach(function (i) { %> <li><%= i.title %></li> <% }}; %> </ul> <% } else { %> empty <% } %>`.

8. Что такое *partials* в *EJS* и как их подключить?

Partials – это частичные шаблоны, например шапка или подвал, которые повторно используются на разных страницах. Подключение выполняется строкой `<%- include('partials/header') %>`. В проекте это *header.ejs* и *footer.ejs*.

9. Как в *EJS* передать массив объектов и отрендерить его в таблицу?

Массив передается в `res.render('index', { items })`, а в шаблоне строки таблицы формируются циклом `items.forEach` внутри `<tbody>`, где каждое поле выводится через `<%= item.title %>`. Именно так устроен *index.ejs*.

10. Как реализовать *middleware*, которое выполняется только для определенного маршрута?

Функцию указывают перед обработчиком маршрута: `router.get('/add', requireAuth, pages.addForm)`. Также можно подключить ее по префиксу: `app.use('/admin', checkAdmin)`.

11. Как отличить *middleware* для статики (*express.static*) от других?

express.static – встроенный *middleware*, который по пути запроса ищет файл в указанной папке и отдает его, а при отсутствии файла вызывает `next()`. Остальные *middleware* работают с логикой запроса и не обращаются к файловой системе.

12. Как реализовать *middleware* для проверки авторизации по сессии?

Middleware читает `req.session.user` (после подключения *express-session*) и при его отсутствии делает `res.redirect('/login')`, иначе вызывает `next()`. В проекте вместо сессии проверяется параметр `req.query.auth`.

13. Какая разница между *res.render()* и *res.send()*?

Метод `res.render()` подставляет данные в шаблон и отправляет готовый *HTML*, он нужен для страниц. Метод `res.send()` отправляет переданное значение как есть (строку, объект, буфер), его используют для простых ответов и *API*.

14. Как обработать ошибку в асинхронном *middleware* без *try/catch*?

В *Express 5* отклоненный промис автоматически передается в обработчик ошибок. В *Express 4* используют обертку, которая вызывает `.catch(next)`, например `const wrap = (fn) => (req, res, next) => fn(req, res, next).catch(next)`, либо пакет *express-async-errors*.

15. Как в *EJS* задать базовый шаблон (*layout*)?

Есть два способа: подключать *partials* шапки и подвала в каждой странице (как в проекте) или использовать пакет *express-ejs-layouts*, в котором общий файл *layout.ejs* содержит `<%- body %>`.

16. Где лучше определять *middleware* – до или после маршрутов?

Общие *middleware* (логирование, парсинг тела, авторизация) определяют до маршрутов, иначе они не успеют сработать. Обработчик 404 и обработчик ошибок определяют после всех маршрутов, чтобы они перехватывали запросы, которые никто не обработал.

17. Как с помощью *middleware* перехватить и изменить *req.body*?

После *express.json()* или *express.urlencoded()* подключают собственную функцию, которая изменяет *req.body*, например обрезает пробелы: *req.body.title = req.body.title.trim(); next();*.

18. Можно ли использовать несколько *middleware* последовательно для одного маршрута?

Да. Функции перечисляются через запятую или массивом, и каждая вызывает *next()*: *app.post('/add', requireAuth, validate, controller)*.

19. Как в *EJS* получить доступ к переменной внутри частичного шаблона?

Partial получает все данные родительского шаблона автоматически, включая *res.locals*. Дополнительные значения передаются вторым аргументом: *include('partials/card', { item })*. Так в *header.ejs* используются *title*, *user* и *authQuery*.

20. Как реализовать *middleware*, добавляющее общий заголовок?

app.use((req, res, next) => { res.setHeader('X-Powered-By', 'VisualDSL'); next(); }); – заголовок добавится ко всем ответам, подключенным после этого *middleware*.

21. Какие *middleware* предоставляет *Express* для обработки форм?

express.urlencoded({ extended: true }) разбирает тела формата *application/x-www-form-urlencoded* и заполняет *req.body*. Для *JSON* используется *express.json()*. В проекте подключены оба.

22. Как отключить стандартный *HTML*-вывод ошибок и вернуть *JSON*?

Нужно определить собственный обработчик ошибок и ответить через *res.status(500).json({ error: '...' })*. Тогда встроенный обработчик *Express*, показывающий стек в *HTML*, не вызывается. В проекте так отвечают маршруты */api/*.

23. Как в *EJS* подключить внешний *CSS* и *JS*?

Папка со статикой подключается через *app.use('/static', express.static('public'))*, а в шаблоне указывается абсолютный путь: *<link rel="stylesheet" href="/static/css/style.css">*. Относительные пути лучше не использовать, они ломаются на вложенных маршрутах.

24. Как реализовать *middleware*, измеряющее время выполнения запроса?

Фиксируют время начала, а после события *finish* ответа вычисляют разницу: `res.on('finish', () => console.log(Date.now() - start))`. В проекте это *middlewares/logger.js*, где используется *process.hrtime.bigint()* для точности.

25. Как реализовать защиту от *CSRF* с помощью *middleware* (теоретически)?

Сервер выдает случайный токен, сохраняет его в сессии и вставляет скрытым полем в форму. *Middleware* сравнивает токен из тела запроса с сохраненным и отклоняет запрос при несовпадении. Дополнительно используют *cookie* с *SameSite*. Готовое решение – пакет *csurf* или его аналоги.

26. Как в *EJS* использовать функцию-*helper*?

Функцию кладут в *app.locals*: `app.locals.formatDate = (iso) => new Date(iso).toLocaleString('ru-RU')`. В шаблоне ее вызывают как обычную: `<%= formatDate(item.createdAt) %>`. В проекте так сделано на главной и детальной страницах.

27. Почему *middleware* для статики обычно подключается до всех маршрутов?

Запросы к стилям, скриптам и картинкам должны обслуживаться сразу, не проходя через авторизацию, парсеры и логику маршрутов. Это ускоряет ответ и исключает случайный перехват файлов обработчиком 404.

28. Как заблокировать запросы с определенного *IP*-адреса?

Middleware проверяет *req.ip* по списку запрещенных адресов и отвечает `res.status(403).send('Forbidden')`, иначе вызывает *next()*. За прокси необходимо включить `app.set('trust proxy', true)`, чтобы *req.ip* содержал адрес клиента.

29. Как в *EJS* отрендерить переменную без *HTML*-экранирования?

Используется тег `<%- value %>` вместо `<%= value %>`. Это опасно, так как введенный пользователем *HTML* будет исполнен, что приводит к *XSS*. Тег `<%-` допустим только для доверенной разметки, например для *include*.

30. Как организовать *error-handling middleware*, которая ловит все ошибки приложения?

Обработчик с четырьмя параметрами регистрируют последним, после всех маршрутов и обработчика 404. Ошибки из синхронного кода попадают в него автоматически, из асинхронного – через `next(err)`. Внутри логируют ошибку и отправляют страницу *500.ejs* или *JSON*.

5 Выводы

В ходе работы создано *Express*-приложение с шаблонизатором *EJS*, тематика которого соответствует проекту *VisualDSL*. Реализованы *middleware* логирования с выводом журнала в таблицу (время, событие, метод, *URL*, статус и другие поля) и имитации авторизации, страницы списка, деталей и

добавления элемента, а также отдельные страницы ошибок 404 и 500. Данные хранятся в массиве в памяти и используются как *HTML*-страницами, так и *REST API*.

Правильность работы подтверждена 117 автоматическими тестами и скриншотами страниц, а покрытие серверного кода по строкам составило 100 %. Практическим результатом стало понимание того, что порядок подключения *middleware* определяет поведение приложения: статика и разбор тела запроса подключаются до маршрутов, а обработчики 404 и 500 – после них.